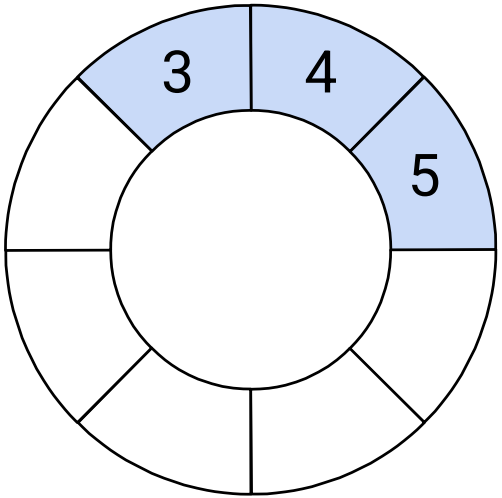
```
[3, 4, 5]
```

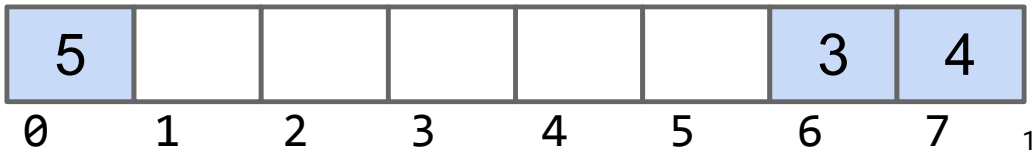


Lecture 8 (Lists 4)

ArrayLists

CS61B, Fall 2026 @ UC Berkeley

Josh Hug and Manuel Sabin



AList vs. DLList

IntList, SLList, and DLList Improvements	
#1	Rebranding: `IntList` → `IntNode`
#2	Bureaucracy: `SLList`
#3	Access Control: `public` → `private`
#4	Nested Class: Bringing `IntNode` into `SLList`
#5	Caching: Saving `size` as an `int`.
#6	Generalizing: Adding a `sentinel` node to allow representation of the empty list.
#7	Looking back: `.last` and `.prev` allow fast `removeLast`
#8	Sentinel upgrade: Avoiding special cases with `sentBack` or circular list.
#9	Generics: DLList can store anything

One unfixable downside of DLLists: Can't access items in the middle quickly.

One approach: Use an array instead.

CC BY NC SA

AList Problems to Address

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100];
        size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

AList problems to address today:

- Expanding our array to support unlimited elements.
- Generics: making AList able to store anything.
- Optimizing AList speed.

AList Improvements	
#1	Fast to access items anywhere in the list

CC BY NC SA

Resizing Array Theory

Lecture 8, CS61B, Fall 2026

CC BY NC SA

Resizing ArrayLists

- **Resizing Array Theory**
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

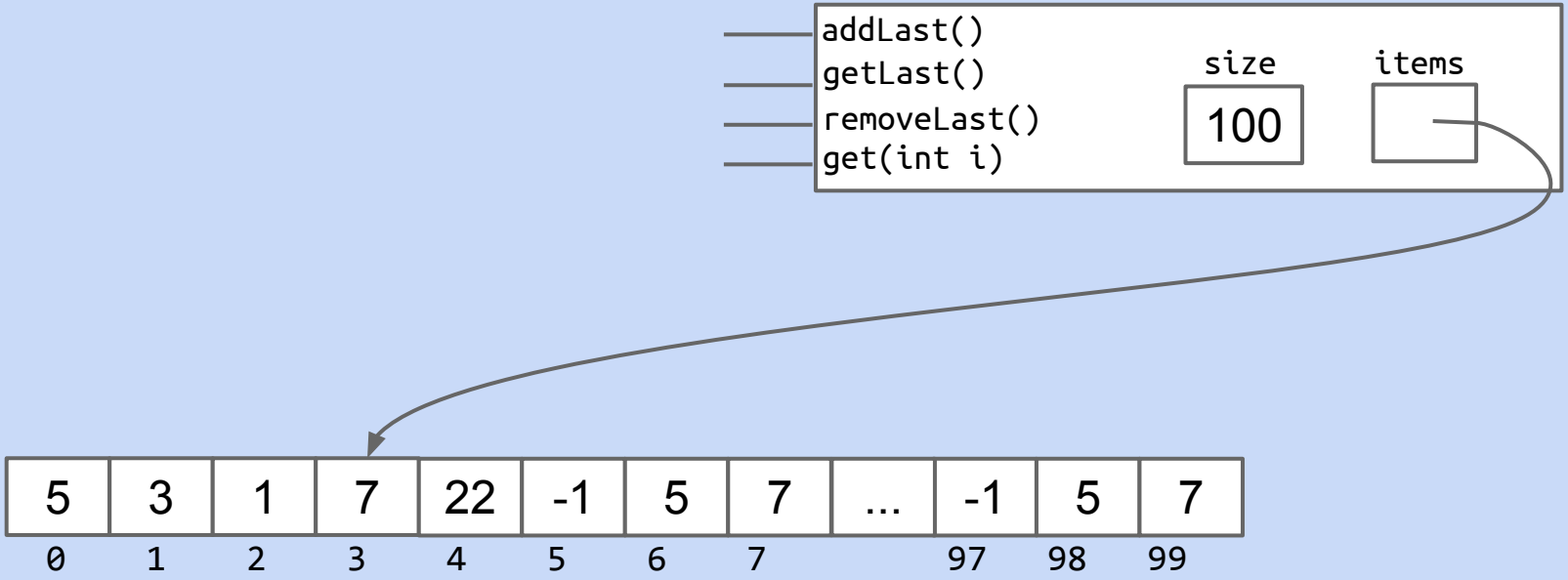
Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

The Mighty (?) AList

Key Idea of AList: Store items in the front part of the array, leaving empty space at the end for later items.



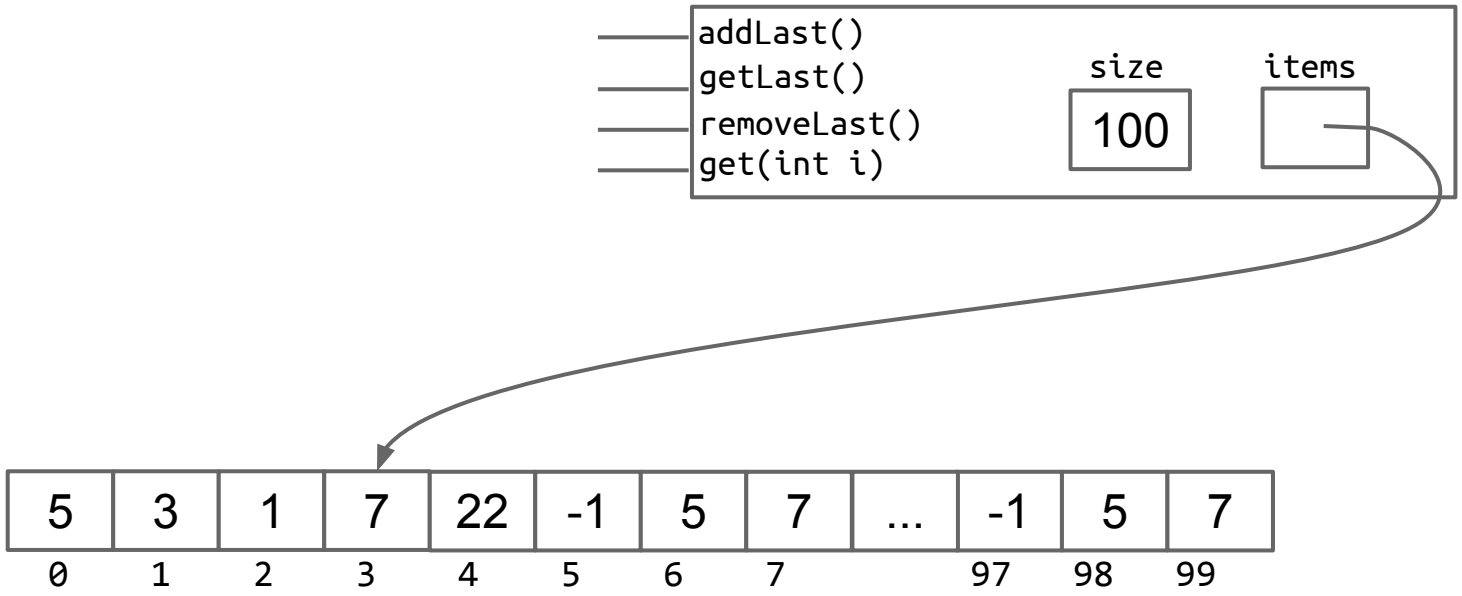
What happens if we insert into the AList above? What should we do about it?

CC BY NC SA

Array Resizing

size==items.length

When the array gets too full, e.g. addLast(11), just make a new array:



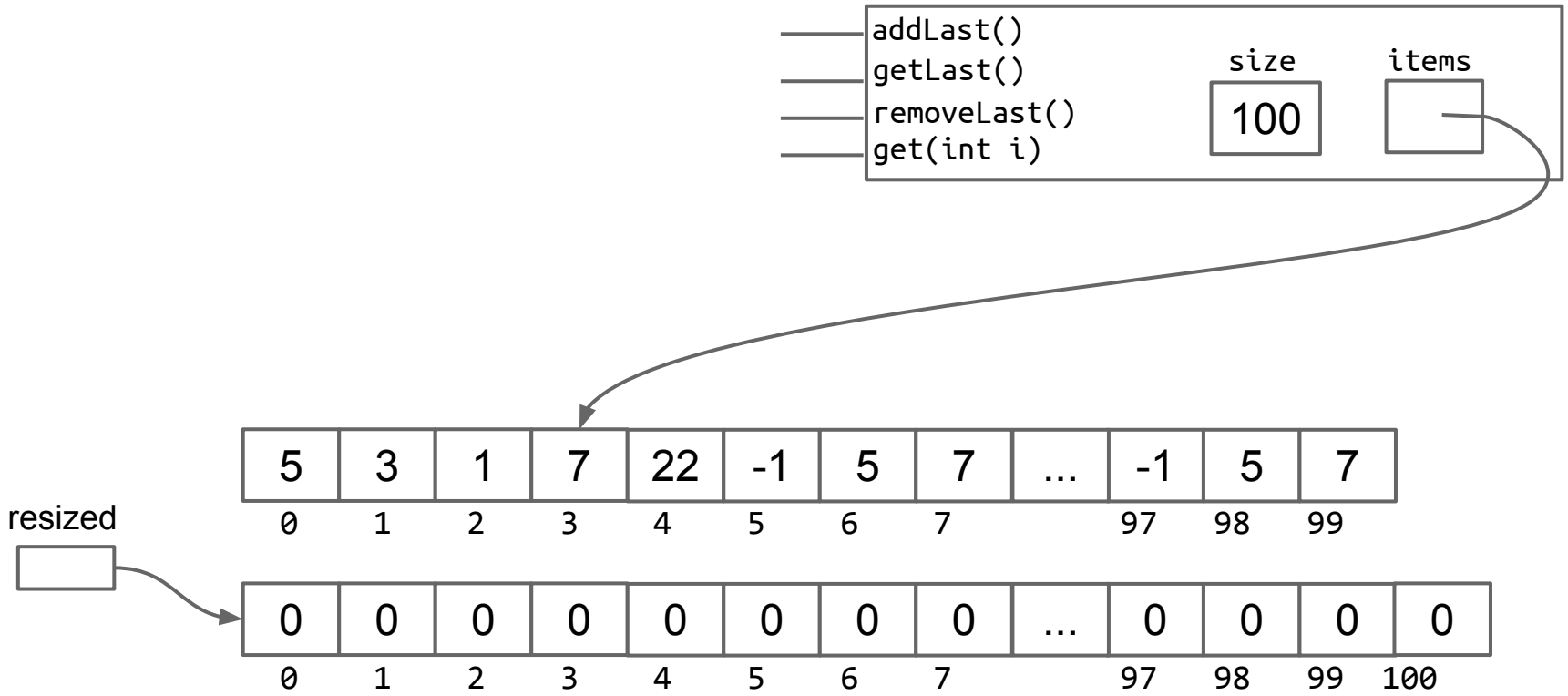
CC BY NC SA

Array Resizing

size==items.length

When the array gets too full, e.g. addLast(11), just make a new array:

- `int[] resized = new int[size+1];`



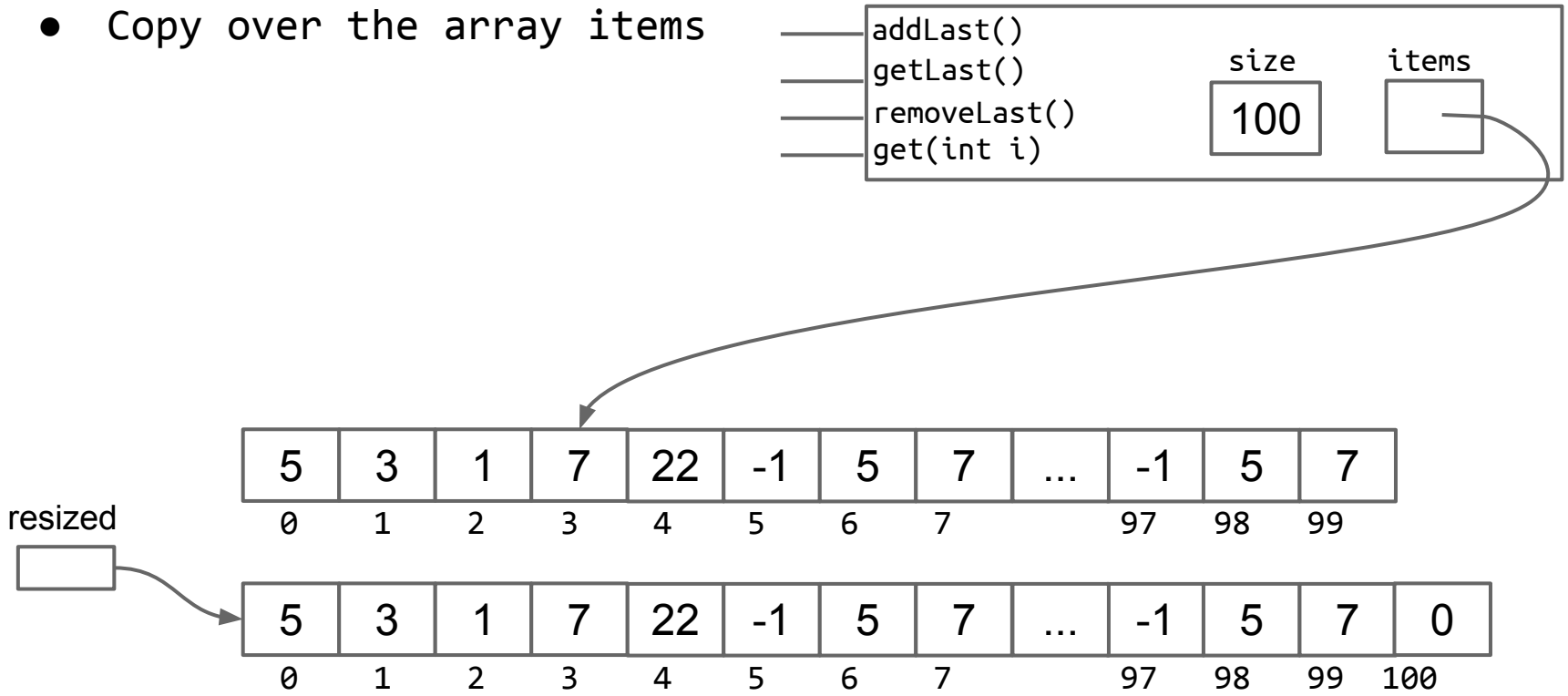
CC BY NC SA

Array Resizing

size==items.length

When the array gets too full, e.g. addLast(11), just make a new array:

- `int[] resized = new int[size+1];`
- Copy over the array items



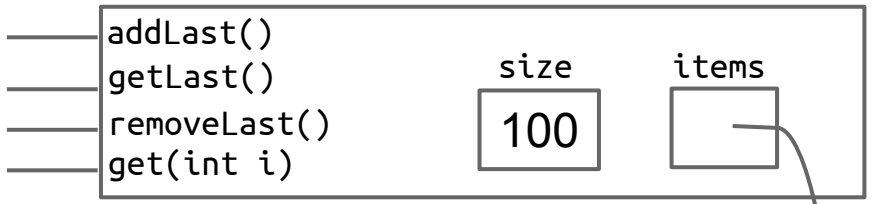
CC BY NC SA

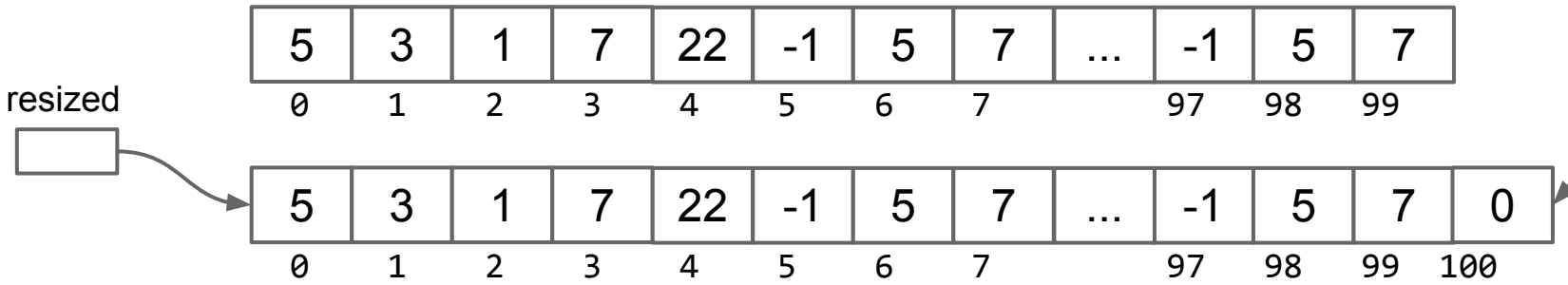
Array Resizing

size==items.length

When the array gets too full, e.g. addLast(11), just make a new array:

- `int[] resized = new int[size+1];`
- `Copy over the array items`
- `items = resized;`



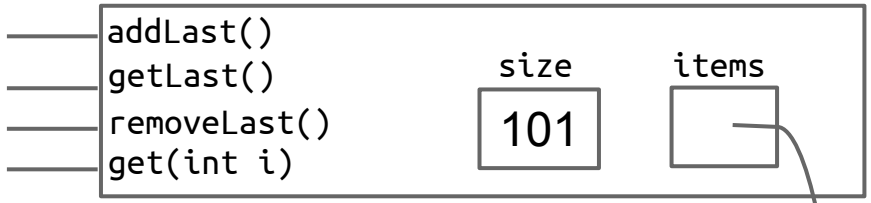


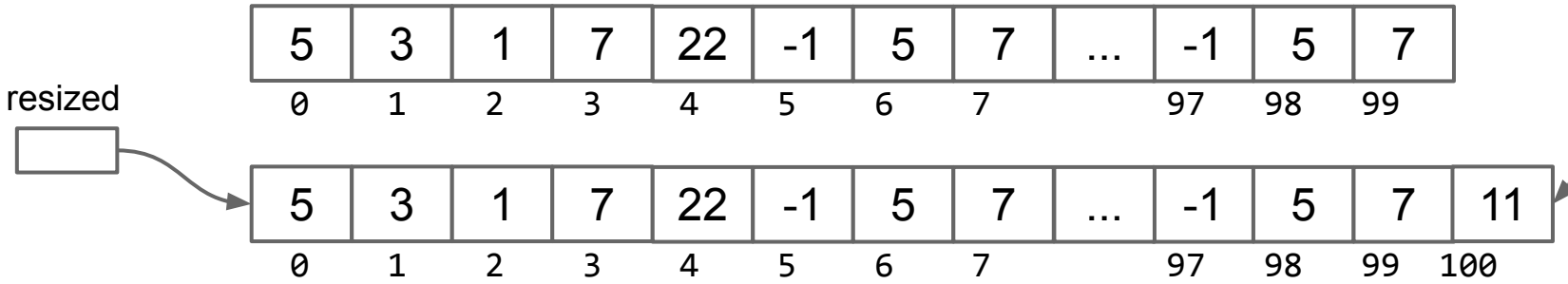
Array Resizing

size==items.length

When the array gets too full, e.g. addLast(11), just make a new array:

- `int[] resized = new int[size+1];`
- `Copy over the array items`
- `items = resized;`
- `items[size] = 11;`
- `size +=1;`





CC BY NC SA

Array Resizing

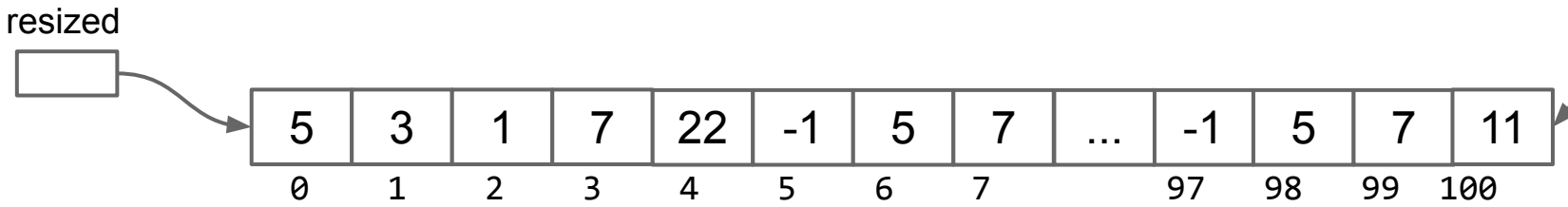
size==items.length

When the array gets too full, e.g. addLast(11), just make a new array:

- `int[] resized = new int[size+1];`
- `Copy over the array items`
- `items = resized;`
- `items[size] = 11;`
- `size +=1;`

We call this process “resizing”





CC BY NC SA

Quick Note on Arraycopy

Two ways to copy arrays:

- Item by item using a loop.
- Using arraycopy. Takes 5 parameters:
  - Source array
  - Start position in source
  - Target array
  - Start position in target
  - Number to copy

```
System.arraycopy(items, 0, a, 3, 2);
```

```
(In Python): a[3:5] = items[0:2]
```

arraycopy is (likely to be) faster, particularly for large arrays. More compact code.

- Code is (arguably) harder to read.
- Recommendation: Don't use on project 2 (it's a trap for reasons we'll see later today!).

CC BY NC SA

Resizing Array Implementation

Lecture 8, CS61B, Fall 2026

CC BY NC SA

Resizing ArrayLists

- Resizing Array Theory
- **Resizing Array Implementation**
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

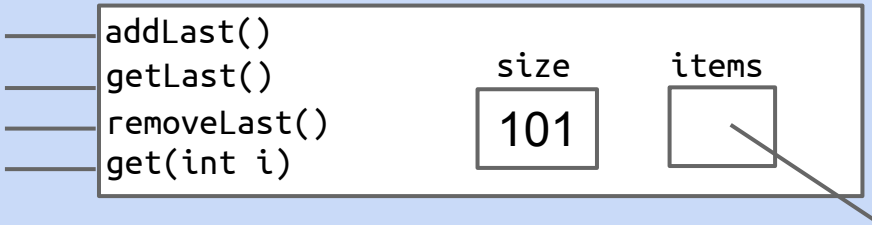
Implementation: hugcode.com/cloud (optional!)

Implement the resizing capability in `addLast`

```
public void addLast(int x) {
   if (__________________________) {
       int[] resized = new int[size + 1];
       for (________________________) {
           ______________ = ______________;
       }
       __________ = resized;
   }
   items[size] = x;
   size += 1;
}
```

Exercise: www.hugcode.com/61b/lecs/lec8

Feel free just to think about this in your head if you'd like!



CC BY NC SA

Coding Demo: Resizing Array

AList.java

```
public class AList {
   /** Resizes the underlying array to the target capacity. */
   private void resize(int capacity) {
       int[] resized = new int[capacity];
       for (int i = 0; i < size; i++) {
           resized[i] = items[i];
       }
       items = resized;
    }

    /** Inserts x into the back of the list. */
    public void addLast(int x) {
        if (size == items.length) {
            resize(size + 1);
        }
        items[size] = x;
        size += 1;
    }
}
```

CC BY

Resizing Array Code

```
public void addLast(int x) {
  if (size == items.length) {
      int[] resized = new int[size + 1];
      for (int i = 0; i < size; i++) {
          resized[i] = items[i];
      }
      items = resized;
  }
  items[size] = x;
  size += 1;
}
```

Works

```
private void resize(int capacity) {
    int[] resized = new int[capacity];
    for (int i = 0; i < size; i++) {
        resized[i] = items[i];
    }
    items = resized;
}

public void addLast(int x) {
  if (size == items.length) {
    resize(size + 1);
  }
  items[size] = x;
  size += 1;
}
```

Much Better

Easier to read and understand.
Easier to test the correctness of each function.

CC BY NC SA

Runtime Analysis (Warmup)

Lecture 8, CS61B, Fall 2026

CC BY NC SA

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- **Runtime Analysis (Warmup)**
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

Runtime and Space Usage Analysis: hugcode.com/mango (Also attendance!)

Suppose we have a full array of size 100. If we call `addLast` two times, how many **total** array memory boxes will we need to create and fill (for just these 2 calls)?

A. 0
B. 2
C. 102
D. 203
E. 10,302



Bonus question: What is the maximum number of array boxes that Java will track at any given time? Assume that "garbage collection" happens immediately when all references to an object are lost.

```
public void resize(int capacity) {
   int[] resized = new int[capacity];
   for (int i = 0; i < size; i++) {
       resized[i] = items[i];
   }
   items = resized;
}

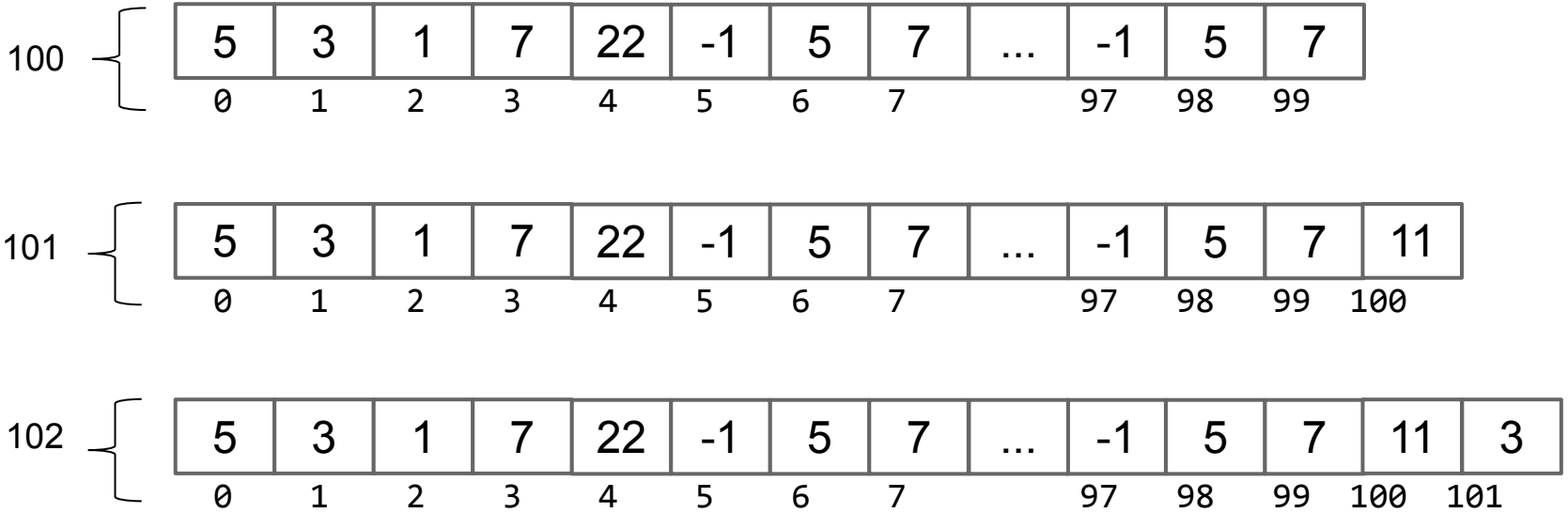
public void addLast(int x) {
  if (size == items.length) {
    resize(size + 1);
  }
  items[size] = x;
  size += 1;
}
```

CC BY NC SA

Array Resizing

Resizing twice requires us to create and fill 203 total memory boxes.

- Bonus answer: Most boxes at any one time is 203.
- When the second `addLast` is done, we are left with 102 boxes.



CC BY NC SA

Runtime Analysis

Lecture 8, CS61B, Fall 2026

CC BY NC SA

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- **Runtime Analysis**
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

Demo: Speed Testing the ArrayList

```
jug ~/.../speedtest
$ java SpeedTestSLList

Total runtime: 3 ms
```

SpeedTestSLList.java

```
public class SpeedTestSLList {
  public static void main(String[] args) {
    long startTime = System.currentTimeMillis();

    SLList<Integer> L = new SLList<>();
    int i = 0;
    while (i < 100000) {
      L.addFirst(i);
      i = i + 1;
    }

    long endTime = System.currentTimeMillis();
    System.out.println("Total runtime: "
                       + (endTime - startTime) + " ms");
   }
}
```

Adding 100,000 items to a new SLList is very fast (order of milliseconds).

cc BY NC SA

Demo: Speed Testing the ArrayList

```
jug ~/.../speedtest
$ java SpeedTestAList

Total runtime: 602 ms
```

SpeedTestAList.java

```java
public class SpeedTestAList {
  public static void main(String[] args) {
    long startTime = System.currentTimeMillis();

    AList L = new AList();
    int i = 0;
    while (i < 100000) {
      L.addLast(i);
      i = i + 1;
    }

    long endTime = System.currentTimeMillis();
    System.out.println("Total runtime: "
                + (endTime - startTime) + " ms");
   }
}
```

Adding 100,000 items to a new AList is slow (order of seconds).

CC BY NC SA

Runtime and Space Usage Analysis: hugcode.com/hats

Suppose we have a full array of size 100. If we call addLast until size = 1000, roughly how many total array memory boxes will we need to create and fill?

A. 1,000
B. 500,000
C. 1,000,000
D. 500,000,000,000
E. 1,000,000,000,000



Bonus question: What is the maximum number of array boxes that Java will track at any given time? Assume that "garbage collection" happens immediately when all references to an object are lost.

```
public void resize(int capacity) {
   int[] resized = new int[capacity];
   for (int i = 0; i < size; i++) {
       resized[i] = items[i];
   }
   items = resized;
}

public void addLast(int x) {
  if (size == items.length) {
    resize(size + 1);
  }
  items[size] = x;
  size += 1;
}
```

CC BY NC SA

Runtime and Space Usage Analysis

Suppose we have a full array of size 100. If we call addLast until size = 1000, roughly how many total array memory boxes will we need to create and fill?

B. 500,000

```
public void resize(int capacity) {
   int[] resized = new int[capacity];
   for (int i = 0; i < size; i++) {
       resized[i] = items[i];
   }
   items = resized;
}
```

Going from capacity 100 to 101: 101

From 101 to 102: 102

…

From: 999 to 1000: 1000

We'll be doing a lot of this after the midterm.

Total array boxes created/copied: 101 + 102 + … + 1000

Since sum of 1 + 2 + 3 + … + N = N(N+1)/2, sum(101, …, 1000) is close to 500,000.

See: http://mathandmultimedia.com/2010/09/15/sum-first-n-positive-integers

- Or see hidden slide that follows this one.

CC BY NC SA

Runtime and Space Usage Analysis

Since sum of 1 + 2 + 3 + … + N = N(N+1)/2, sum(3, …, 100) is close to 5050.

Rough intuition: Form pairs that all sum to N+1:

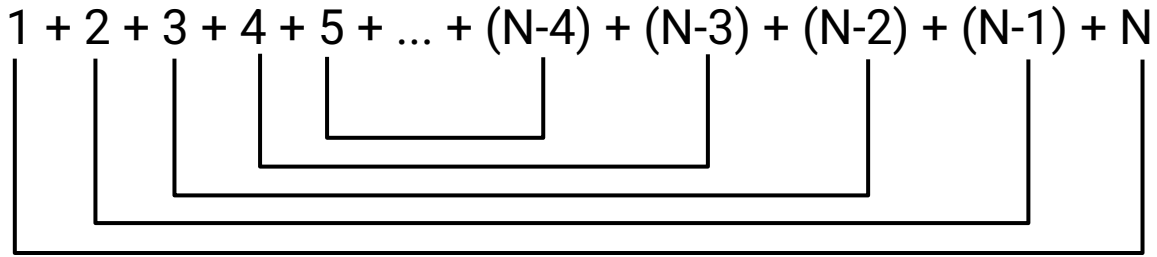
N + 1

(N-1) + 2

(N-2) + 3

...

There are N/2 such pairs.



CC BY NC SA

SLList vs. AList Timing Tables

Below we see the time to insert N items for SLLists and ALists.

```
jug ~/.../speedtest
$ java SpeedTestSLListTable

N               Time (ms)
10000           1
100000          3
1000000         71
10000000        875
```

```
jug ~/.../speedtest
$ java SpeedTestAListTable

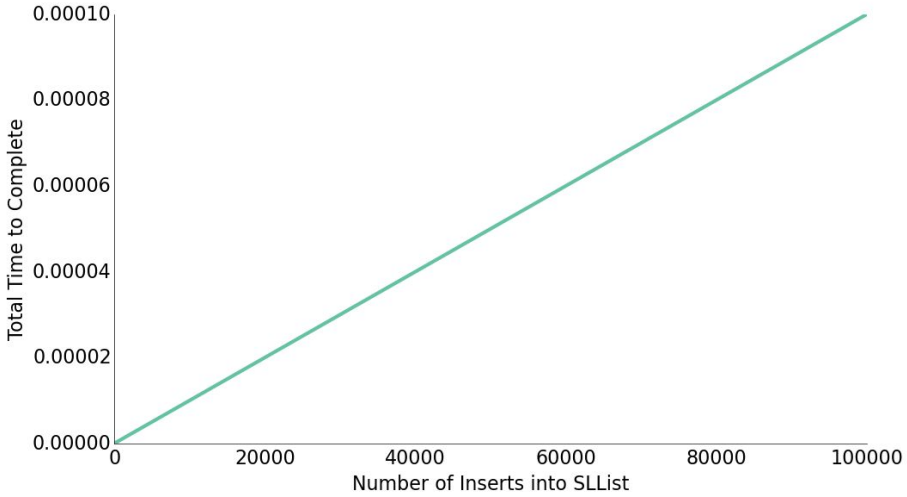
N               Time (ms)
10000           19
100000          903
1000000         70227
```

CC BY NC SA

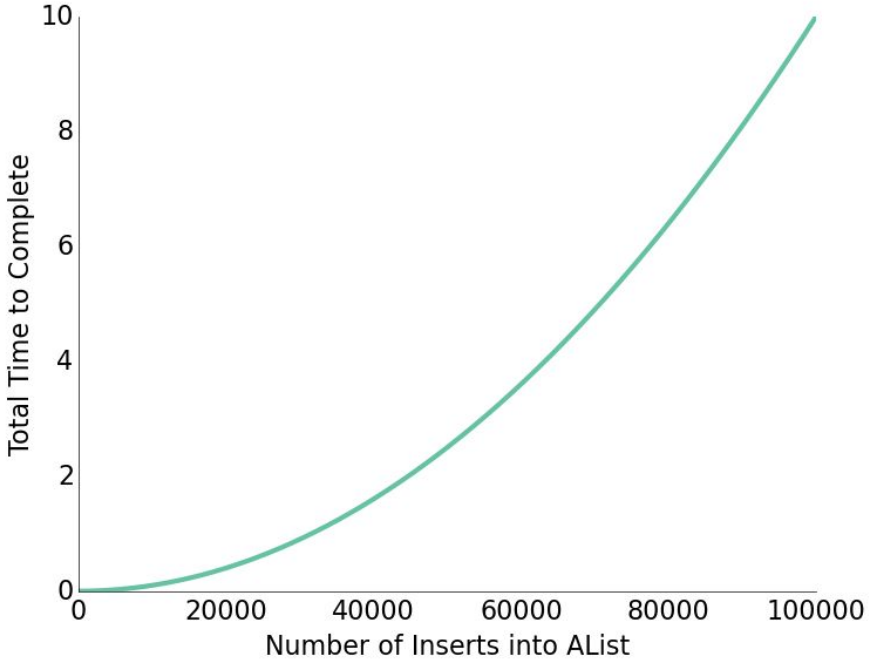
Resizing Slowness (Theoretical Plots)

Inserting 100,000 items requires roughly 5,000,000,000 new containers.

- Computers operate at the speed of GHz (do billions of things per second).
- No huge surprise that 100,000 items took roughly seconds.



Note: Insert here is addFirst



cc BY NC SA

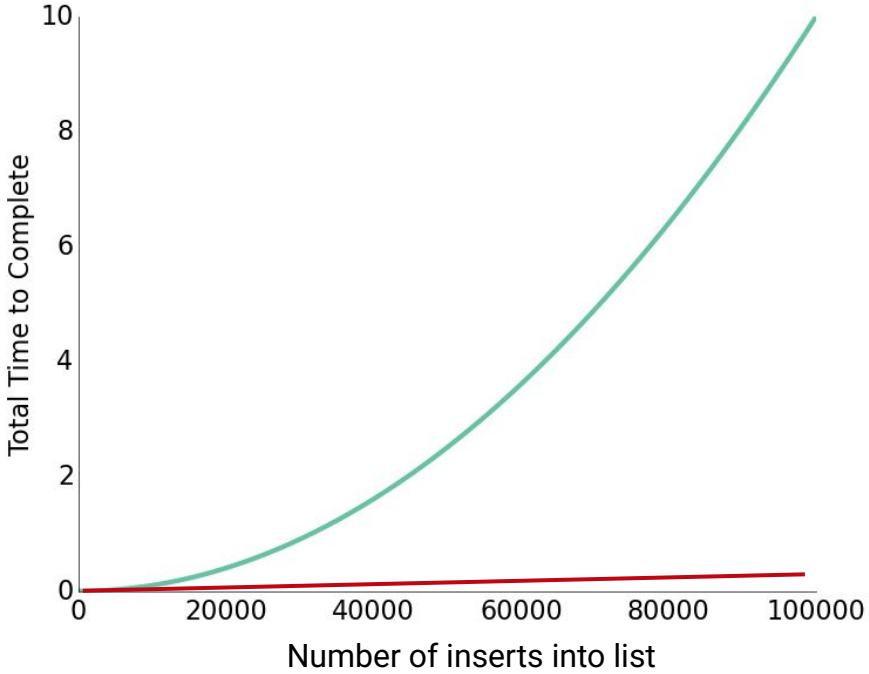
Resizing Slowness (Theoretical Plots)

Inserting 100,000 items requires roughly 5,000,000,000 new containers.

- Computers operate at the speed of GHz (do billions of things per second).
- No huge surprise that 100,000 items took roughly seconds.

The same graphs from the previous slide, placed on top of each other.

Red line = SLList
Teal line = AList



cc BY NC SA

Better Resizing Strategy

Lecture 8, CS61B, Fall 2026

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- **Better Resizing Strategy**

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

CC BY NC SA

Fixing the Resizing Performance Bug

How do we fix this?

```
public void resize(int capacity) {
   int[] resized = new int[capacity];
   for (int i = 0; i < size; i++) {
       resized[i] = items[i];
   }
   items = resized;
}

public void addLast(int x) {
  if (size == items.length) {
    resize(size + 1);
  }
  items[size] = x;
  size += 1;
}
```

CC BY NC SA

Demo: Speed Testing the ArrayList

```
public void addLast(int x) {
    if (size == items.length) {
        resize(size + 1000);
    }
    items[size] = x;
    size += 1;
}
```

```
jug ~/.../speedtest
$ java SpeedTestAList

Total runtime: 6 ms
```

N=100,000

Resizing by 1000 elements instead of 1 seems to speed up adding 100,000 items… (was 602ms)

```
jug ~/.../speedtest
$ java SpeedTestAList

Total runtime: 6097 ms
```

N=10,000,000

…but adding 10,000,000 items is still super slow.

CC BY NC SA

Demo: Speed Testing the ArrayList

```
public void addLast(int x) {
    if (size == items.length) {
        resize(size * 2);
    }
    items[size] = x;
    size += 1;
}
```

```
jug ~/.../speedtest
$ java SpeedTestAList

Total runtime: 2 ms
```

N=100,000

If we double the array capacity every time it's full, then adding 100,000 items is fast…

```
jug ~/.../speedtest
$ java SpeedTestAList

Total runtime: 25 ms
```

N=10,000,000

…and adding 10,000,000 items is also fast.

CC BY NC SA

(Probably) Surprising Fact

Geometric resizing is much faster.

We can't prove this until later. See this video for a more detailed analysis.

Rough intuition: As the array grows larger, we resize less often.

```
public void addLast(int x) {
    if (size == items.length) {
        resize(size + RFACTOR);
    }
    items[size] = x;
    size += 1;
}
```

← Unusably bad.

Great performance. →

This is how the Python list is implemented.

```
public void addLast(int x) {
    if (size == items.length) {
        resize(size * RFACTOR);
    }
    items[size] = x;
    size += 1;
}
```

CC BY NC SA

Performance Problem #2

Suppose we have a very rare situation occur which causes us to:

- Insert 1,000,000,000 items.
- Then remove 990,000,000 items.

Our data structure will execute these operations acceptably fast, but afterwards there is a problem.

- What is the problem?

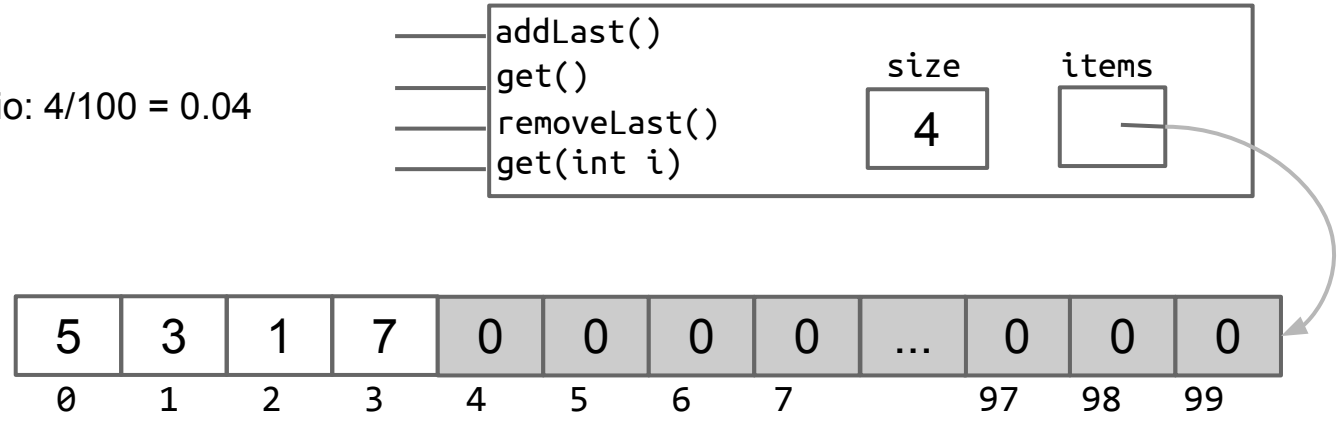
CC BY NC SA

Memory Efficiency

An AList should not only be efficient in time, but also efficient in space.

- Define the “usage ratio” R = size / items.length;
- Typical solution: Halve array size when R < 0.25.
- More details in a few weeks.

Usage ratio: 4/100 = 0.04



Later we will consider tradeoffs between time and space efficiency for a variety of algorithms and data structures.

CC BY NC SA

AList Problems to Address

```
public class AList {
    private int[] items;
    private int size;

    public AList() {
        items = new int[100];
        size = 0;
    }

    public void addLast(int x) {
        items[size] = x;
        size += 1;
    }

    public int get(int i) {
        return items[i];
    }

    public int size() {
        return size;
    }
}
```

AList problems to address today:

- ~~Expanding our array to support unlimited elements~~
- Generics: making AList able to store anything
- Optimizing AList speed

AList Improvements	
#1	Fast to access items anywhere in the list
#2	Allow re-sizing of the underlying array

CC BY NC SA

Generic ArrayLists

Lecture 8, CS61B, Fall 2026

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

CC BY NC SA

Generic ALists (similar to generic SLists)

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[8];
       size = 0;
   }

   public void resize(int capacity) {
      int[] resized = new int[capacity];
          for (int i = 0; i < size; i++) {
            resized[i] = items[i];
          }
      items = resized;
   }

   public int get(int i) {
       return items[i];
   }
   ...
}
```

What would we change to make it generic?

CC BY

Generic ALists (similar to generic SLists): hugcode.com/lemon

```
public class AList {
   private int[] items;
   private int size;

   public AList() {
       items = new int[8];
       size = 0;
   }

   public void resize(int capacity) {
      int[] resized = new int[capacity];
          for (int i = 0; i < size; i++) {
            resized[i] = items[i];
          }
      items = resized;
   }

   public int get(int i) {
       return items[i];
   }
   ...
}
```

```
public class AList<Glorp> {
   private Glorp[] items;
   private int size;

   public AList() {
       items = (Glorp[]) new Object[8];
       size = 0;
   }

   public void resize(int cap) {
      Glorp[] resized = (Glorp[]) new
     Object[cap];
          for (int i = 0; i < size; i++) {
            resized[i] = items[i];
          }
      items = resized;
   }

   public Glorp get(int i) {
       return items[i];
   }
   ...
}
```

CC BY NC SA

Generic ALists (similar to generic SLists)

```
public class AList<Glorp> {
   private Glorp[] items;
   private int size;

   public AList() {
       items = (Glorp[]) new Object[8];
       size = 0;
   }

   public void resize(int cap) {
      Glorp[] resized = (Glorp[]) new
     Object[cap];
          for (int i = 0; i < size; i++) {
            resized[i] = items[i];
          }
      items = resized;
   }

   public Glorp get(int i) {
       return items[i];
   }
   ...
```

When creating an array of references to Glorps:

- `(Glorp[]) new Object[8];`
- Causes a compiler warning, which you should ignore.

Why not just `new Glorp[cap];`

- Will cause a “generic array creation” compilation error.
- For more, here is Angelika Lanker’s 356 page PDF about Java generics FAQ: https://angelikalanger.com/GenericsFAQ/JavaGenericsFAQ.pdf

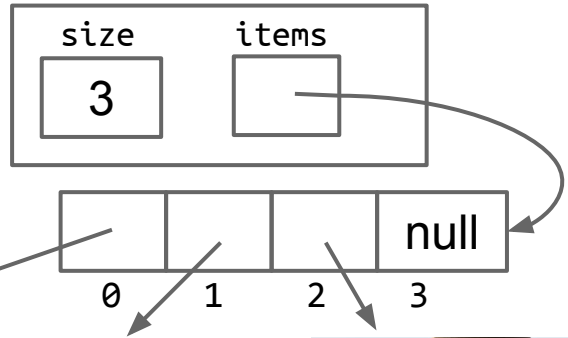
CC BY SA

Nulling Out Deleted Items

Unlike integer based ALists, we actually want to null out deleted items.

- Java only destroys unwanted objects when the last reference has been lost.
- Keeping references to unneeded objects is sometimes called loitering.
- Save memory. Don't loiter.

```
public Glorp removeLast() {
   Glorp returnItem = getLast();
   items[size - 1] = null;
   size -= 1;
   return returnItem;
}
```







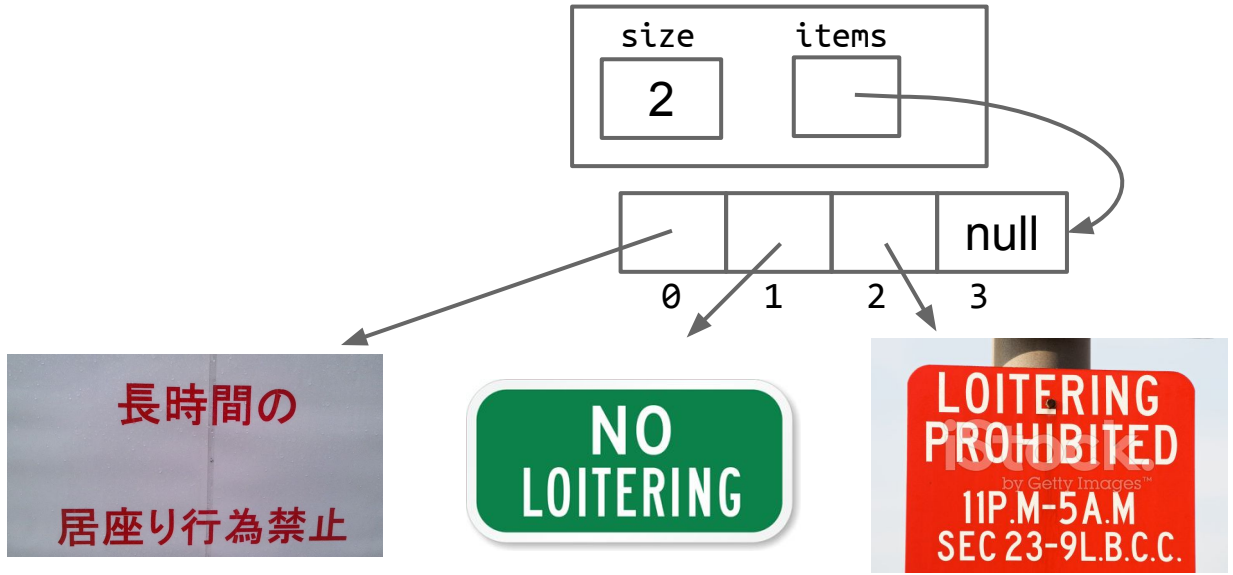


CC BY NC SA

Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

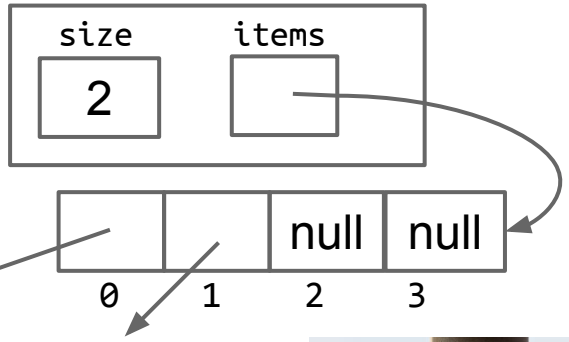


Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

By nulling out items[2], Java is free to destroy the unneeded image from memory, which could be potentially megabytes in size.







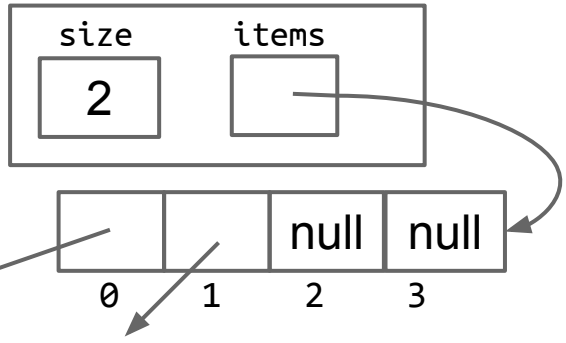


Loitering Example

Changing size to 2 yields a correct AList.

- But memory is wasted storing a reference to the red sign image.

By nulling out items[2], Java is free to destroy the unneeded image from memory, which could be potentially megabytes in size.







CC BY NC SA

AList Problems to Address

```
public class AList<Glorp> {
   private Glorp[] items;
   private int size;

   public AList() {
       items = (Glorp[]) new Object[8];
       size = 0;
   }

   public void resize(int cap) {
      Glorp[] resized = (Glorp[]) new
     Object[cap];
          for (int i = 0; i < size; i++) {
            resized[i] = items[i];
          }
       items = resized;
   }

   public Glorp get(int i) {
       return items[i];
   }
   ...
```

AList problems to address today:

- ~~Expanding our array to support unlimited elements~~
- ~~Generics: making AList able to store anything~~
- Optimizing AList speed

AList Improvements	
#1	Fast to access items anywhere in the list
#2	Allow re-sizing of the underlying array
#3	Generics: AList can store anything

Optimizing ArrayLists

Lecture 8, CS61B, Fall 2026

Resizing ArrayLists

- Resizing Array Theory
- Resizing Array Implementation
- Runtime Analysis (Warmup)
- Runtime Analysis
- Better Resizing Strategy

Generic ArrayLists

Optimizing ArrayLists

- Linear Implementation of Circular Arrays

CC BY NC SA

What are ALists slow at?

```
public class AList<Glorp> {
   private Glorp[] items;
   private int size;

   public AList() {
       items = (Glorp[]) new Object[8];
       size = 0;
   }

   private void resize(int cap) {
       Glorp[] a = (Glorp[]) new Object[cap];
       System.arraycopy(items, 0,
                a, 0, size);
       items = a;
   }

   public Glorp get(int i) {
       return items[i];
   }
   ...
}
```

We build AList to make `get(i)` fast.

Was there anything better about DLList?

```
public class DLList<Pizza> {
   private Node sentinel;
   private int size;

   public class Node {
      public Pizza item;
      public Node prev;
      public Node next;
      ...
   }

   ...
}
```

CC BY NC SA

LinkedLists vs. ArrayLists

We built AList to make `get(i)` fast.

Are ArrayLists strictly better? Is there anything a LinkedList is better at?

- Yes, `addFirst` and `removeFirst` are faster for linked lists.
- Less important: LinkedList operations at ends are always fast at the ends. ArrayList operations sometimes cause a linear time resize (but negligible in practice).

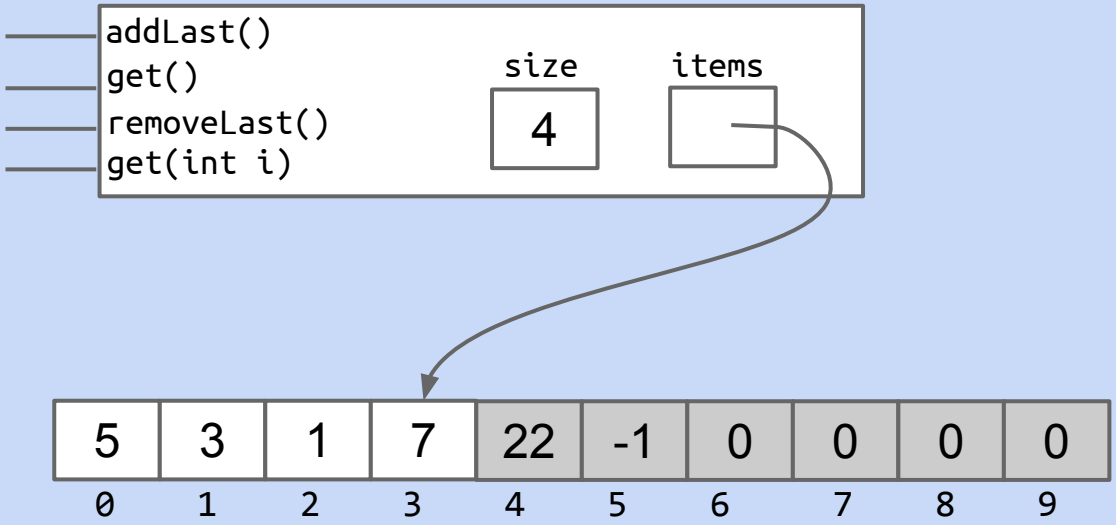
CC BY NC SA

Speeding up Front Operations

For our AList class, `addFirst` or `removeFirst` are going to be slow.

- Have to shift everything to the left.

How might we modify our design so that these front operations are fast?



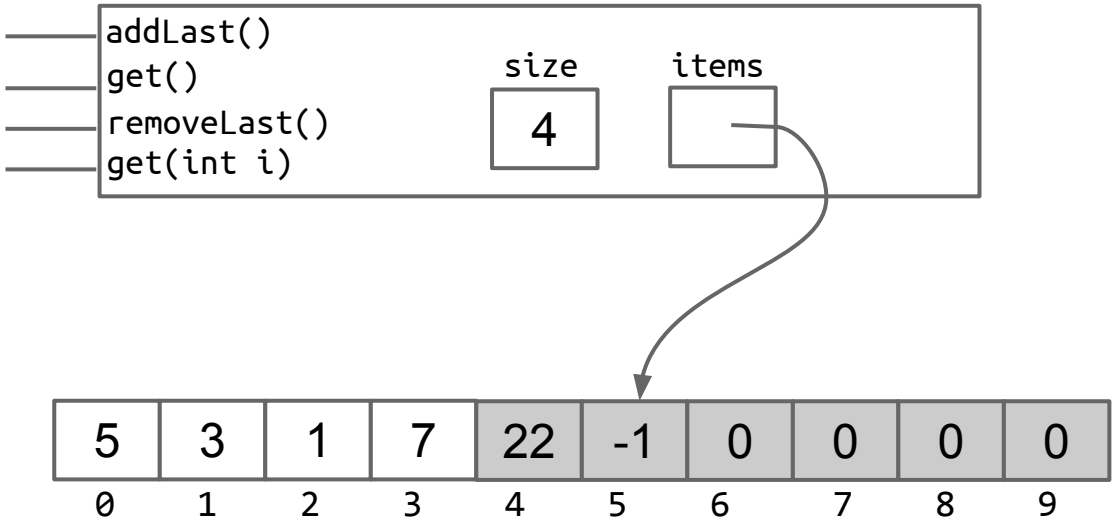
CC BY NC SA

Speeding up Front Operations

```
[5, 3, 1, 7]
```

User's Mental Model





CC BY NC SA

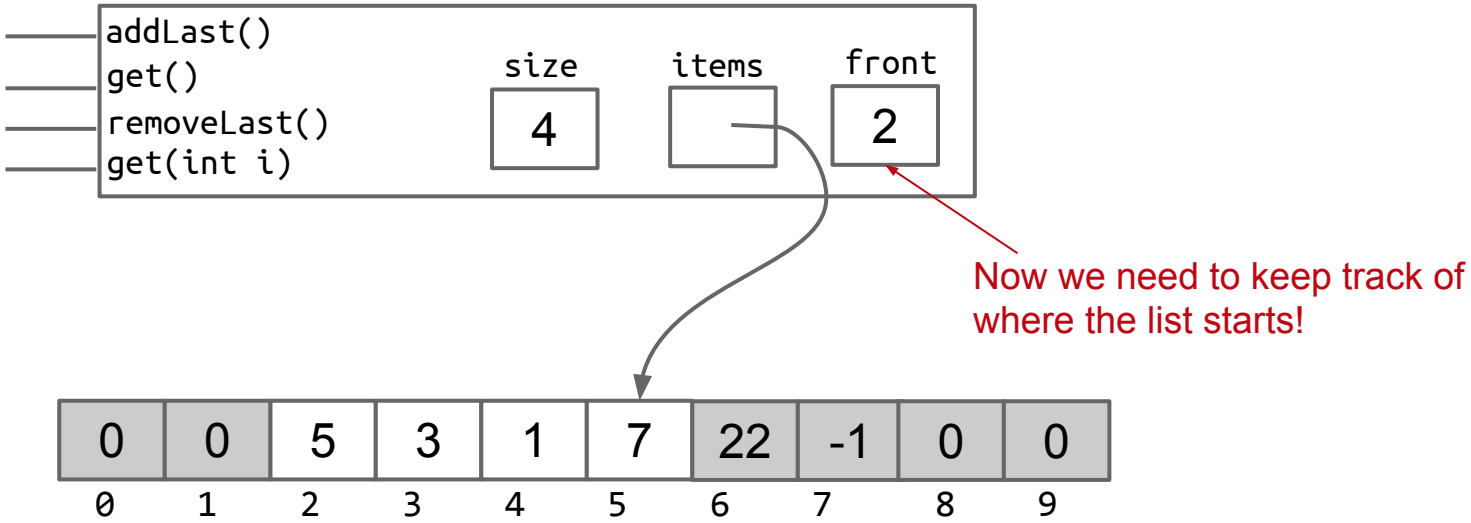
Speeding up Front Operations

What if we moved everything in our list over?

```
[5, 3, 1, 7]
```

User's Mental Model





CC BY NC SA

Speeding up Front Operations

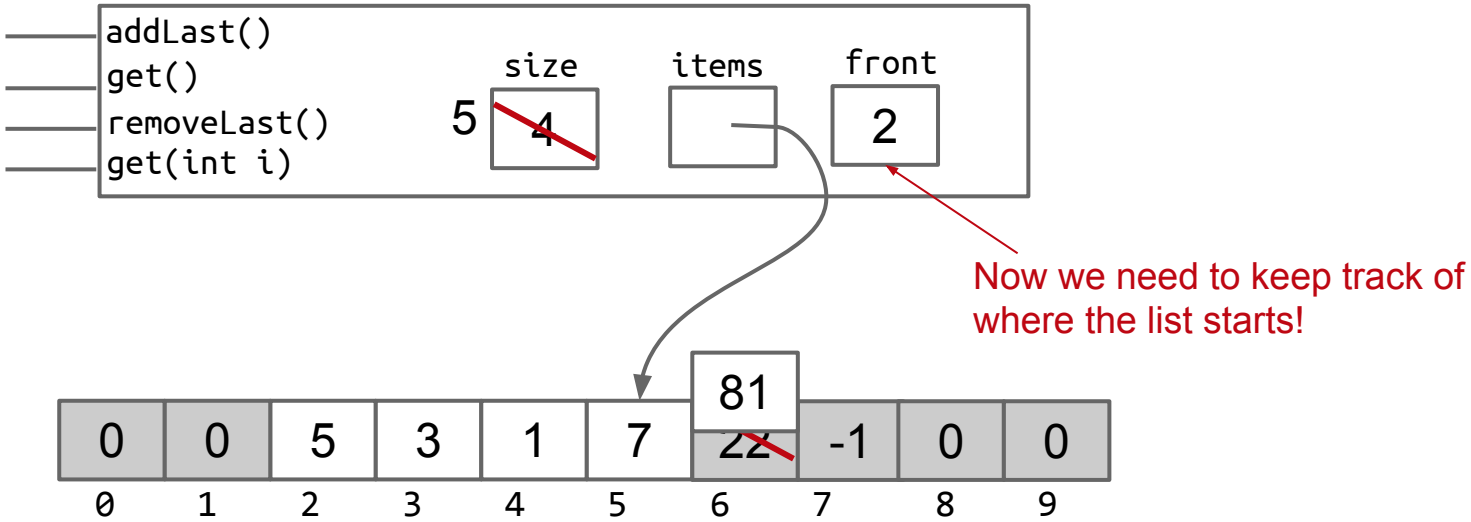
What if we moved everything in our list over?

```
addLast(81);
```

```
[5, 3, 1, 7, 81]
```

User's Mental Model





CC BY NC SA

Speeding up Front Operations

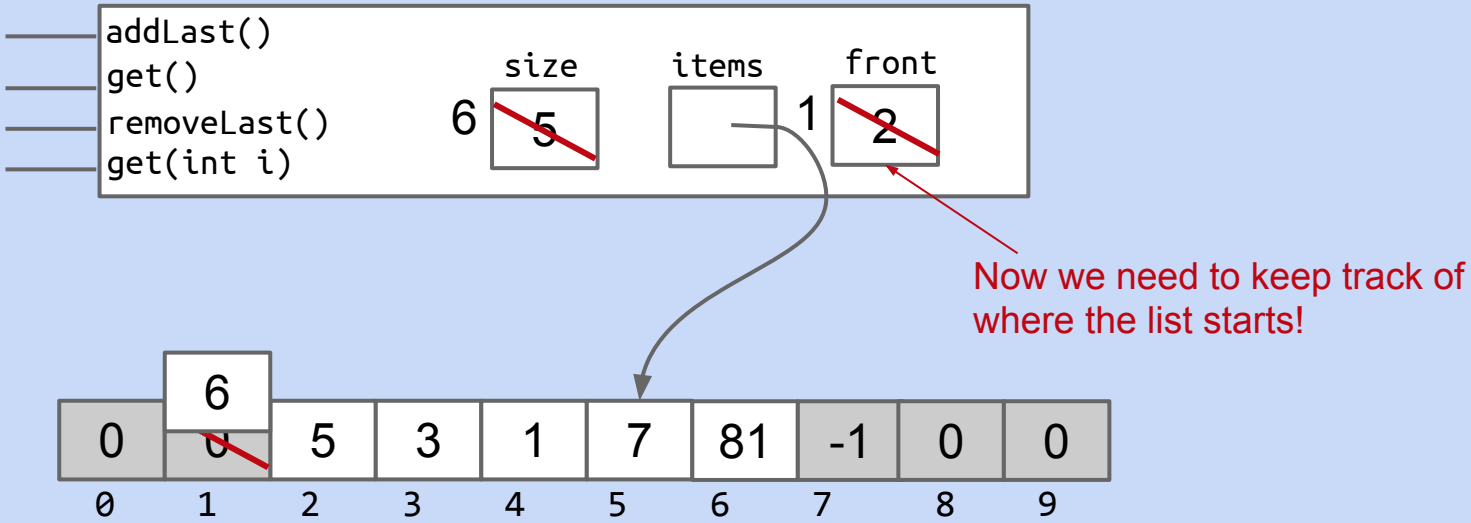
What if we moved everything in our list over?

```
addLast(81);
addFirst(6);
```

```
[6, 5, 3, 1, 7, 81]
```

User's Mental Model





CC BY NC SA

Speeding up Front Operations

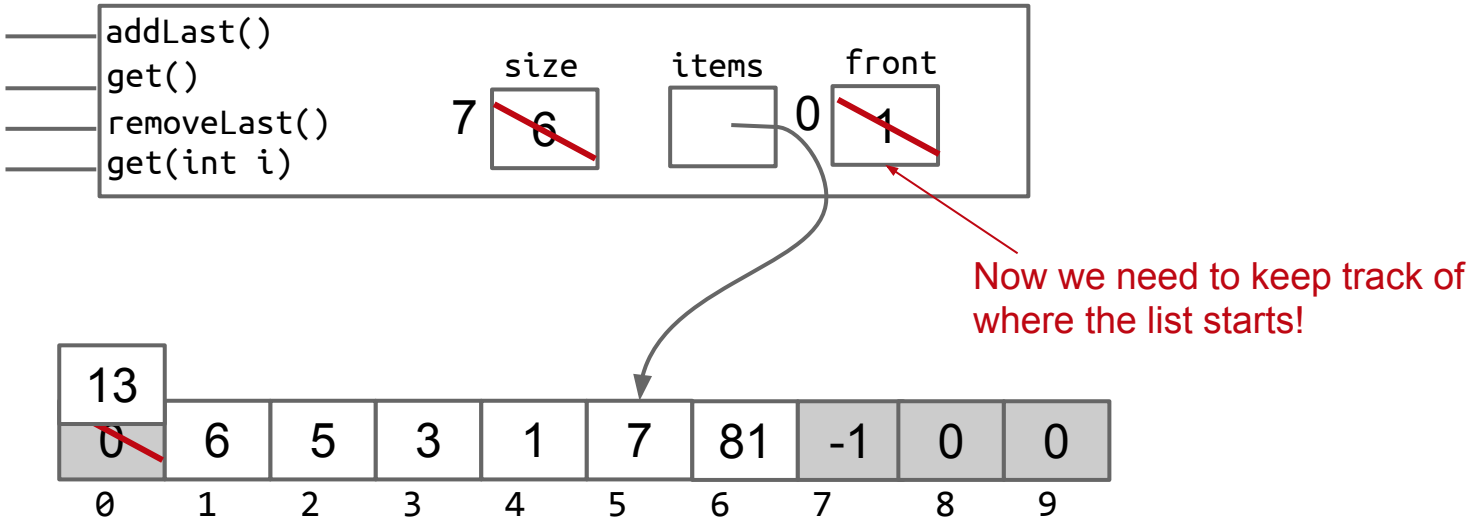
What if we moved everything in our list over?

```
addLast(81);
addFirst(6);
addFirst(13);
```

```
[13, 6, 5, 3, 1, 7, 81]
```

User's Mental Model





CC BY NC SA

Speeding up Front Operations

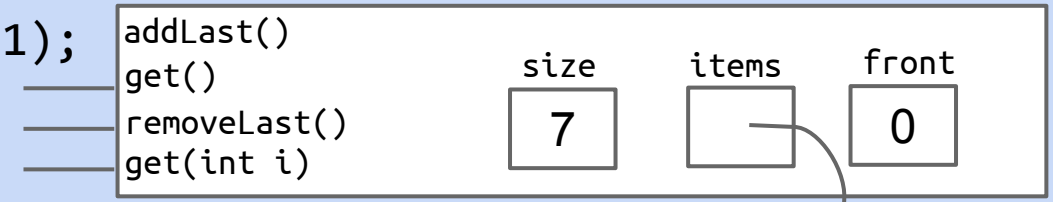
What if we moved everything in our list over?



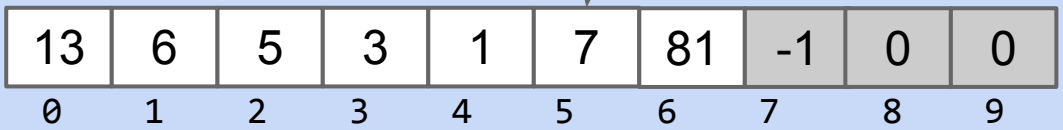
```
addLast(81);
addFirst(6);
addFirst(13);
addFirst(11);
```

```
[11, 13, 6, 5, 3, 1, 7, 81]
```

User's Mental Model



Where can we put the 11?



CC BY NC SA

Speeding up Front Operations

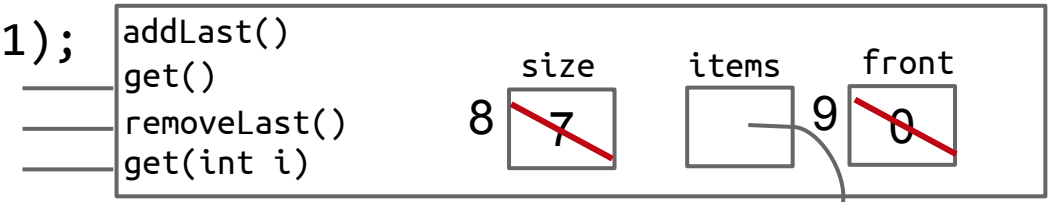
What if we moved everything in our list over?



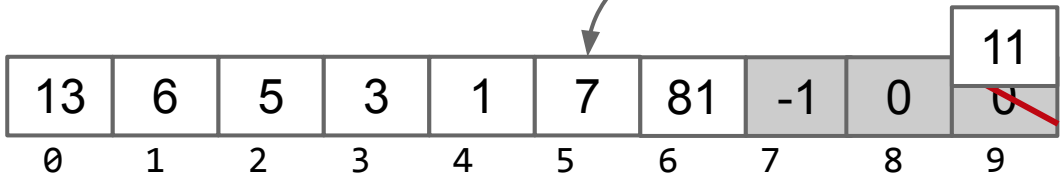
```
addLast(81);
addFirst(6);
addFirst(13);
addFirst(11);
```

```
[11, 13, 6, 5, 3, 1, 7, 81]
```

User's Mental Model



Where can we put the 11?

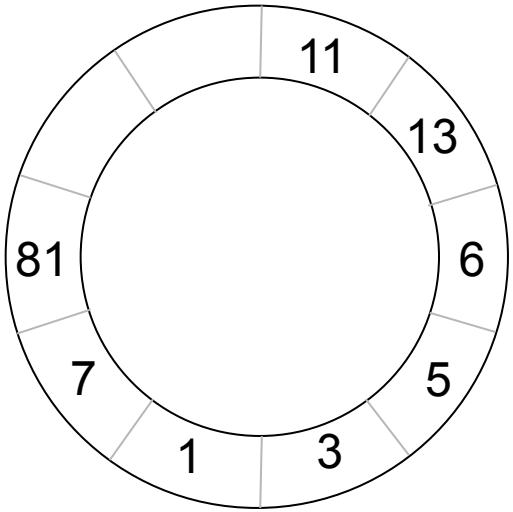


CC BY NC SA

Test Your Understanding

Is the representation unique?

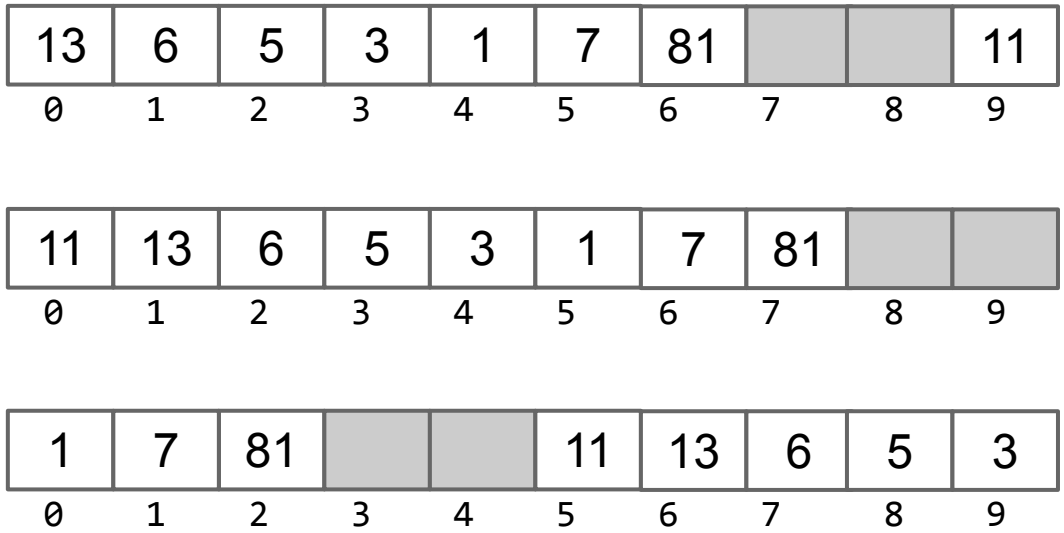
No!



Conceptual Circular Array

```
[11, 13, 6, 5, 3, 1, 7, 81]
```

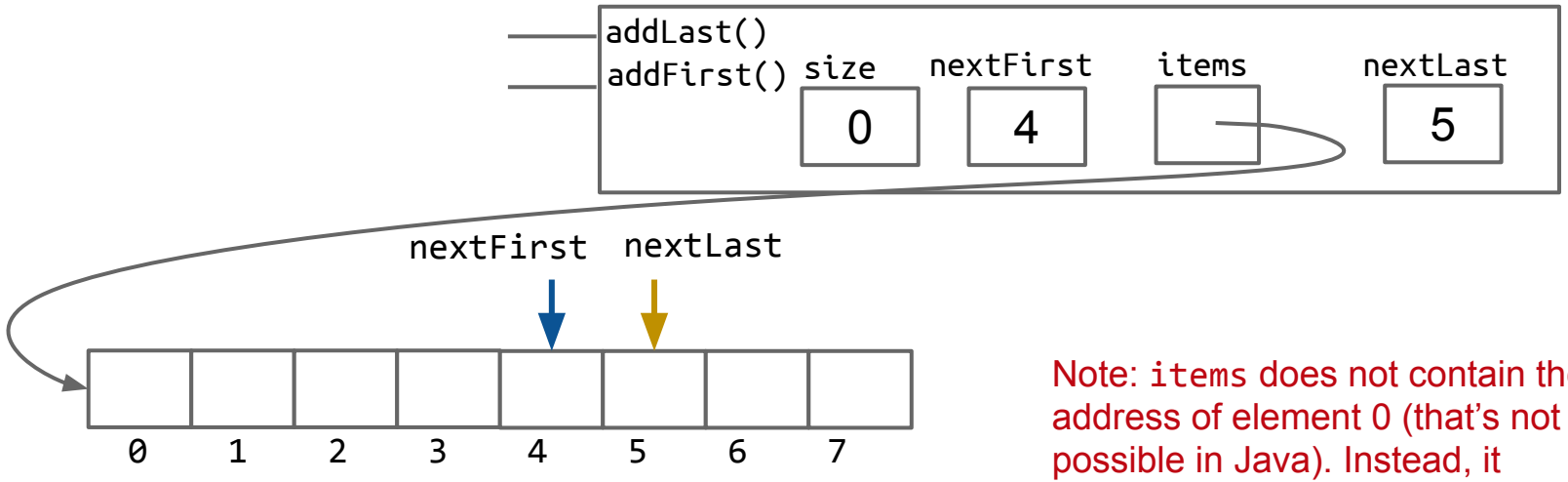
User's Mental Model



Linear Array Demo

Starting from an empty ArrayList:

Conceptual List: []



Note: `items` does not contain the address of element 0 (that's not possible in Java). Instead, it contains the address of the entire array!

You don't have to start your array at 4 or 5. I just picked these arbitrarily.

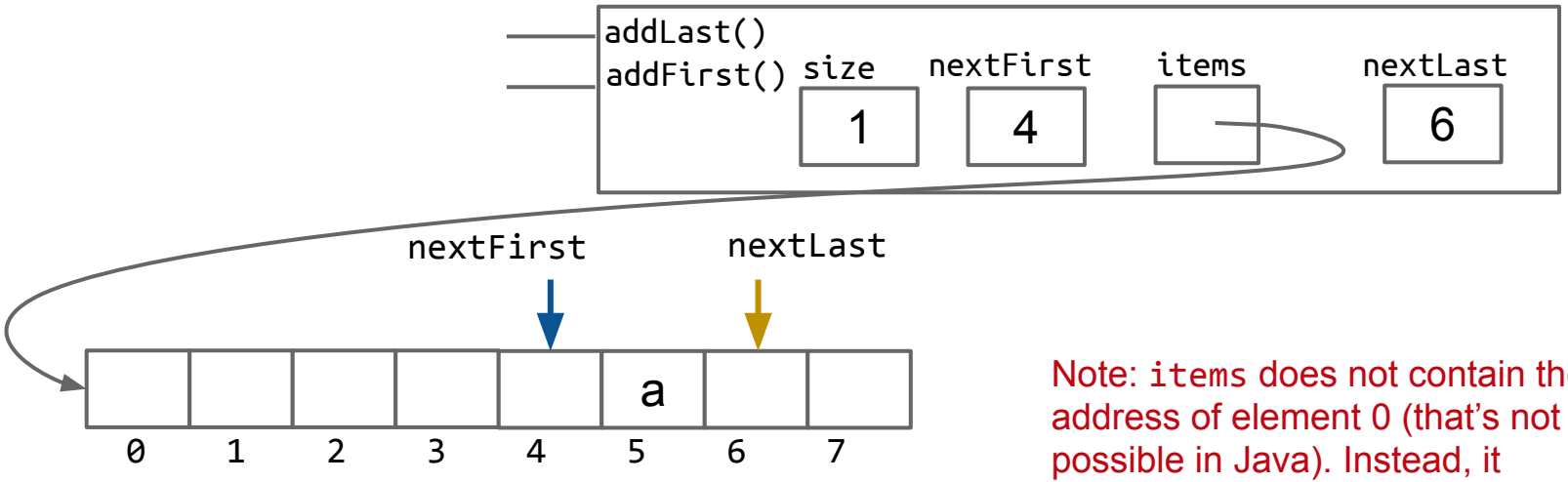
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`

Conceptual List: [a]



Note: `items` does not contain the address of element 0 (that's not possible in Java). Instead, it contains the address of the entire array!

You don't have to start your array at 4 or 5. I just picked these arbitrarily.

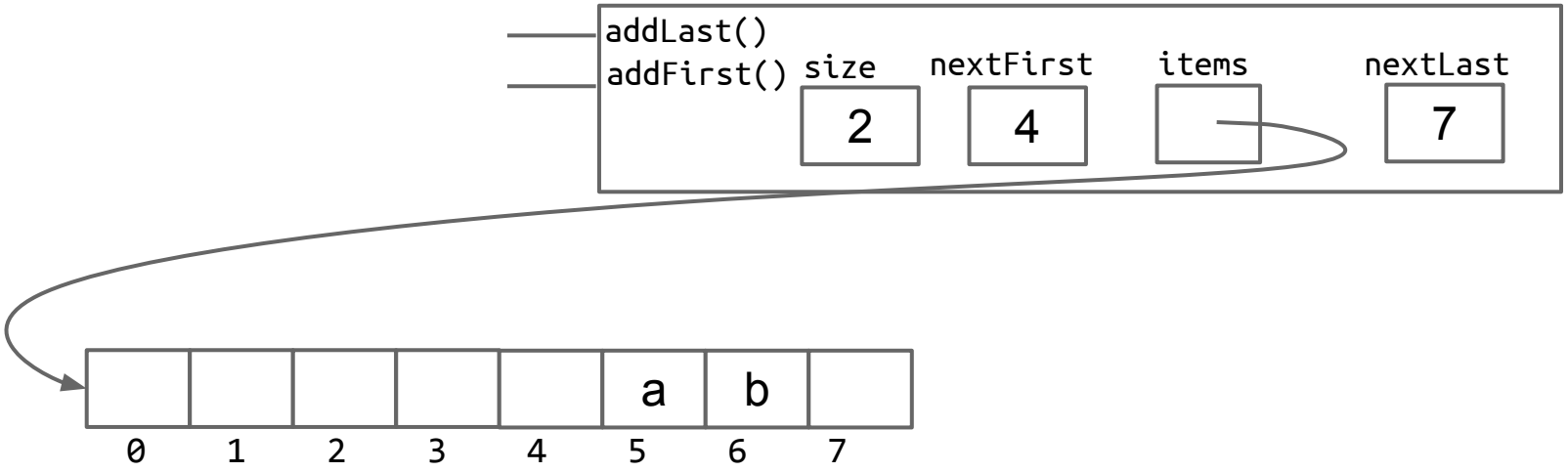
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`

Conceptual List: [a, b]



You don't have to start your array at 4 or 5. I just picked these arbitrarily.

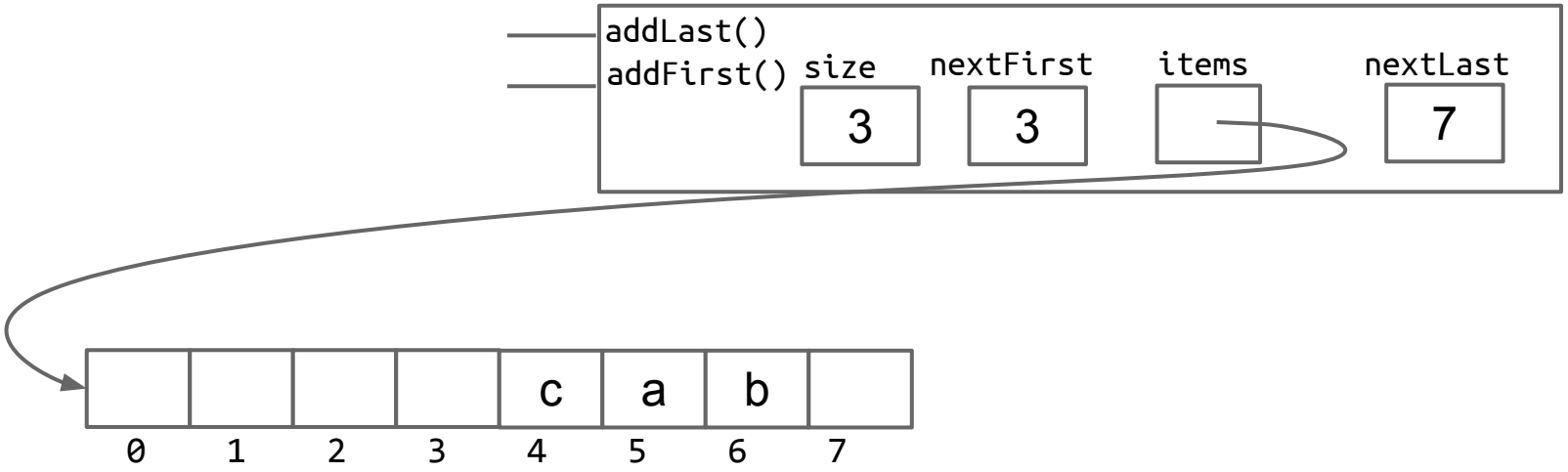
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`

Conceptual List: [c, a, b]



You don't have to start your array at 4 or 5. I just picked these arbitrarily.

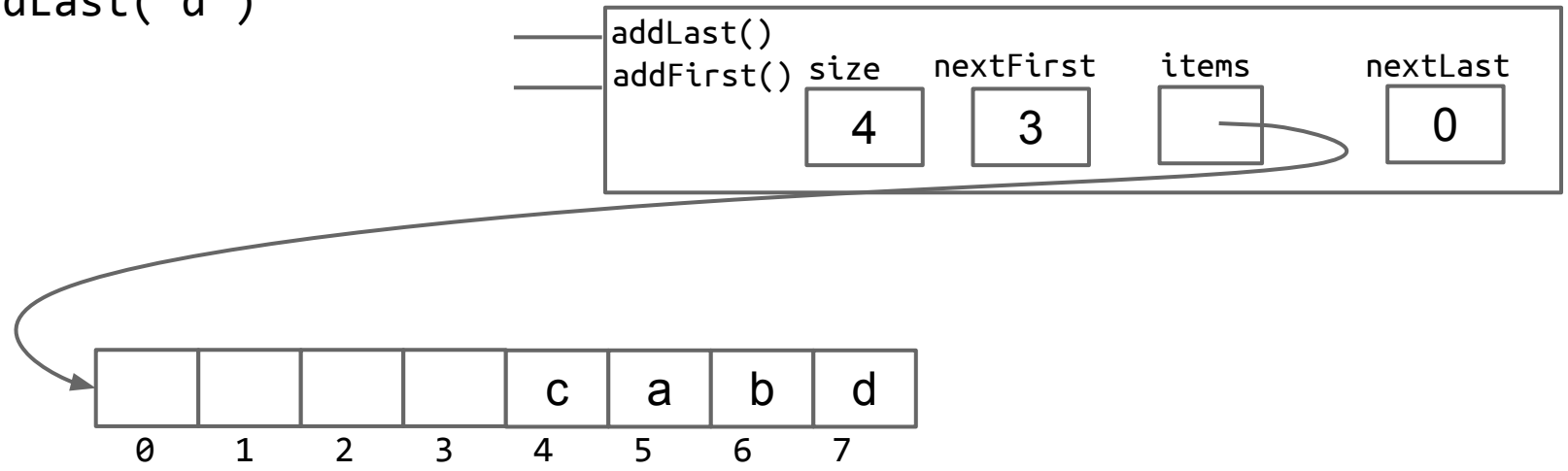
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`

Conceptual List: [c, a, b, d]



You don't have to start your array at 4 or 5. I just picked these arbitrarily.

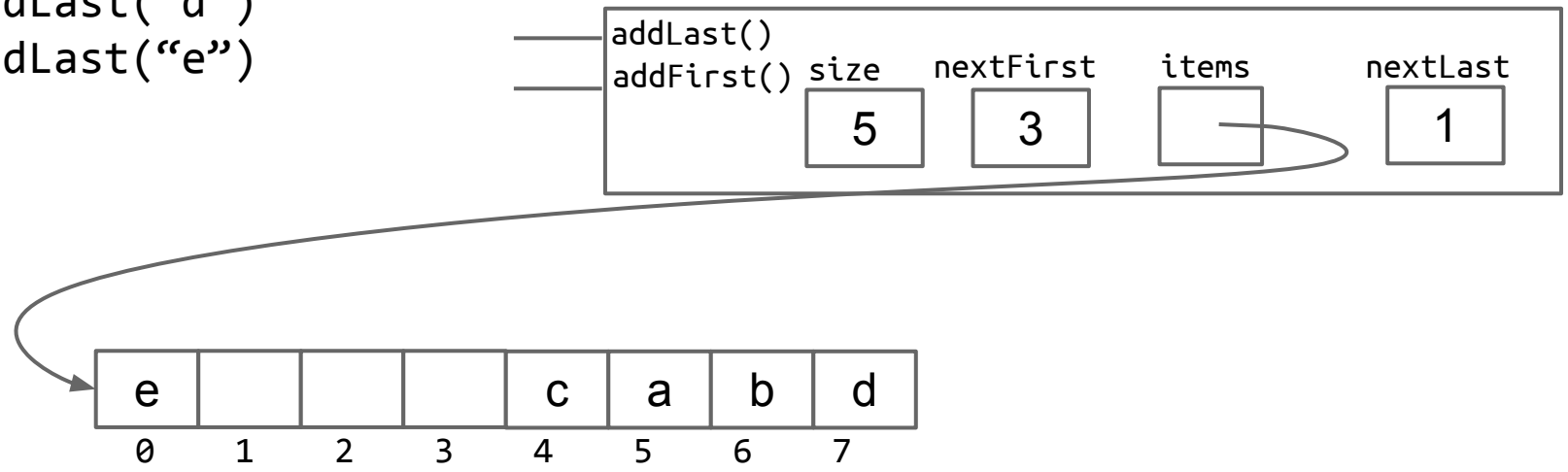
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`

Conceptual List: [c, a, b, d, e]



You don't have to start your array at 4 or 5. I just picked these arbitrarily.

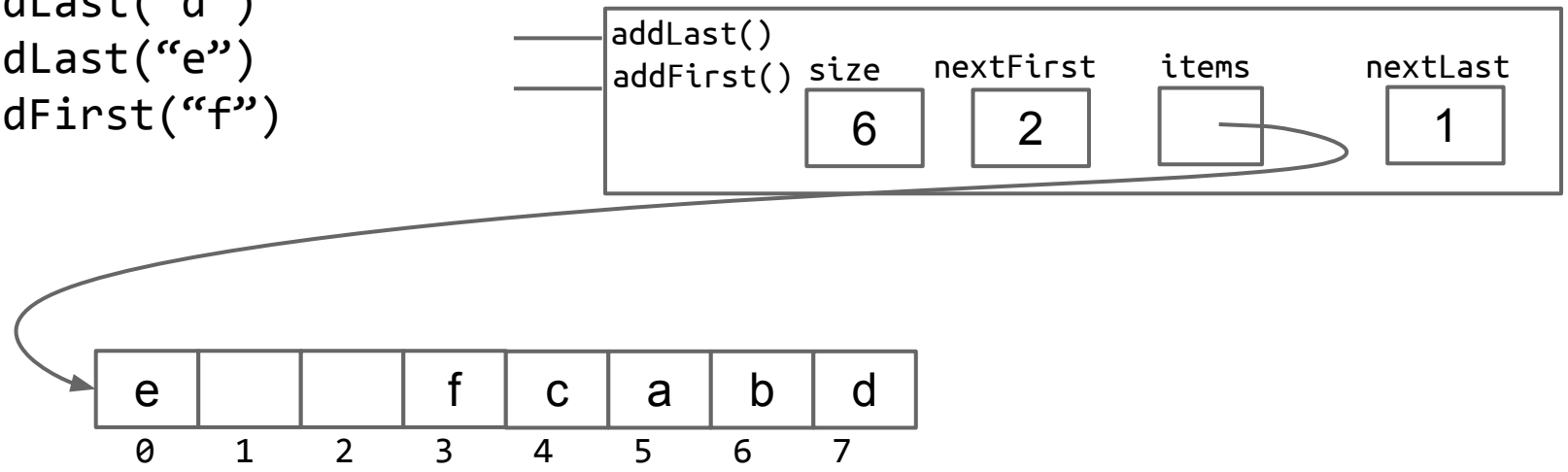
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`

Conceptual List: [f, c, a, b, d, e]



You don't have to start your array at 4 or 5. I just picked these arbitrarily.

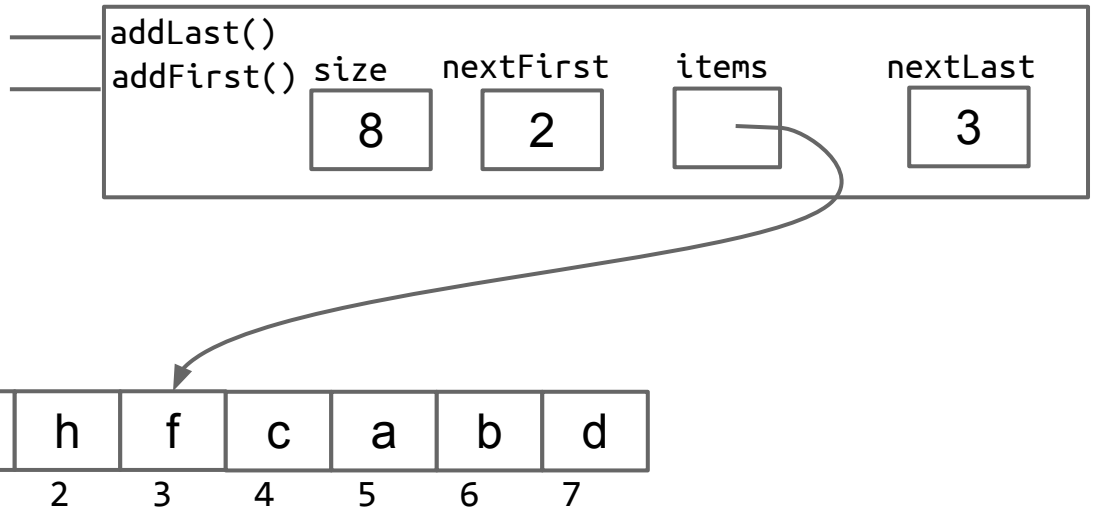
CC BY NC SA

Linear Array Demo

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`
- `addLast("g")`
- `addLast("h")`

Conceptual List: [f, c, a, b, d, e, g, h]



After the next addLast operation, we'll need a bigger array.

CC BY NC SA

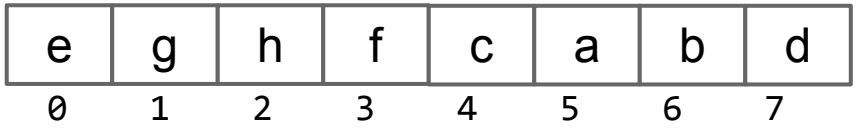
Resizing: You Can Pick Whatever Representation You Prefer for Project 2

Starting from an empty ArrayList:

- `addLast("a")`
- `addLast("b")`
- `addFirst("c")`
- `addLast("d")`
- `addLast("e")`
- `addFirst("f")`
- `addLast("g")`
- `addLast("h")`
- **`addLast("Z")`**

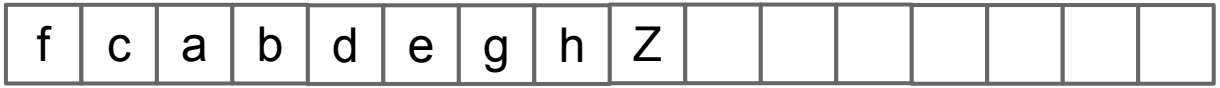
Conceptual List: [f, c, a, b, d, e, g, h, Z]

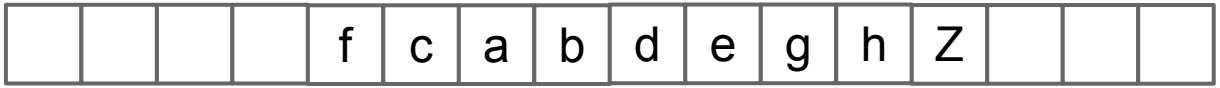
Before `addLast("Z")`:



After `addLast("Z")`: (all of the below are valid choices)







It's your choice what array to use, e.g. all of the three choices above are valid!

CC BY NC SA

Quick Tips on Project 2

Take things a little at a time.

- Writing tons of code all at once is going to lead to misery and only misery.
- If you wrote too much stuff and feel overwhelmed, comment out whatever is unnecessary.

Consider not doing resizing at all until you know your code works without it.

- Resizing is a performance optimization (and is required for full credit).
- Recommendation: Code that uses `arraycopy` can be hard to get right.
- Observation: You might find the `get` method useful in your `resize` method.

If your first try goes badly, don't be afraid to scrap your code and start over.

- There isn't that much code.

CC BY NC SA

AList Problems to Address

```
public class CircularAList<Glorp> {
    private Glorp[] items;
    private int nextFirst;
    private int nextLast;
    private int size;


    public CircularAList() {
        items = (Glorp[]) new Object[8];
        nextFirst = 4;
        nextLast = 5;
        size = 0;
    }

    ...
}
```

Rest of code not shown! This is what you'll be figuring out on Project 2.

AList problems to address today:

- ~~Expanding our array to support unlimited elements~~
- ~~Generics: making AList able to store anything~~
- ~~Optimizing AList speed~~

AList Improvements	
#1	Fast to access items anywhere in the list
#2	Allow re-sizing of the underlying array
#3	Generics: AList can store anything
#4	Make list circular

CC BY NC SA

Note on Python Lists

Python lists do NOT use this circular abstraction.

- `x.insert(0, “hi”)` is slow for Python lists.

Why do you think this is?

CC BY NC SA

Note on Python Lists (My Answer)

Python lists do NOT use this circular abstraction.

- `x.insert(0, “hi”)` is slow for Python lists.

Why do you think this is?

- Insertion at the front of a list is too rare to justify increased code complexity and performance penalty for other operations.

CC BY NC SA

Selecting the Right Data Structure

Users who want fast front operations should use the appropriate data structure.

- `LinkedList` instead of `ArrayList` (in Java).
- `collections.deque` instead of a `list` in Python.
  - Variant of a list with fast front and back operations.

Your project 1 and project 2 solutions are required to have fast front operations (and back operations).

- For your `ArrayDeque`, you are required to use this circular approach.

(Fun (?) fact: Python's built-in Deque class is built using a linked list, not a circular array. For more see https://stackoverflow.com/questions/45134139/why-is-deque-implemented-as-a-linked-list-instead-of-a-circular-array)

CC BY NC SA

To-dos

To-do:

- Project 1B due Wednesday 9/16.
- Project 2 (ArrayDeque) is out; due next Mon 9/21.

CC BY NC SA